MaximizeCRM: A Web-Based Customer Relationship Management System for Dealership Sales Operations

Agrima Agarwal
Department of Computer Science and Engineering
ABES Engineering College, Ghaziabad, Uttar Pradesh, India
Roll No.: 2400320100107
Trainer / Guide: Mrs./Ms. Sakshi Mohania

***Abstract*—Customer Relationship Management (CRM) systems help organizations keep customer data, sales activities and business opportunities in one place. Spreadsheets and disconnected tools lead to duplicated records and poor follow-up. This paper presents MaximizeCRM, a web-based CRM built with a separate frontend and a Spring Boot (Java 17) backend, using PostgreSQL 17 for persistent storage. The system covers users, customers, leads, lead requirements, opportunities, campaigns, products and activities. Access is protected by Spring Security with JSON Web Token (JWT) authentication, frontend and backend communicate through RESTful APIs, and Springdoc OpenAPI documents the endpoints. The deployed interface is configured as a dealership CRM ("Maximize Motors"), with a KPI dashboard, sales pipeline, test-drive scheduling and inventory views. The modular design provides a base for analytics, automated follow-ups and role-based access control.**

***Index Terms*—Customer relationship management, Spring Boot, PostgreSQL, REST API, JWT authentication, Spring Security, web application.**

I. INTRODUCTION

Strong customer relationships are central to business growth. A business interacts with a customer through several stages: an initial enquiry, lead qualification, requirement analysis, a sales opportunity, product discussion and after-sales follow-up. When these stages are tracked by hand or across unconnected tools, information is scattered, records are duplicated and follow-ups are missed.

A CRM system addresses this by providing one platform to store customer information, track potential sales, run campaigns and log business activities [1]. MaximizeCRM is a web-based CRM that organizes these processes in a single application. The frontend provides the user interface, while the backend handles business logic, authentication, API requests and database operations. The backend uses Java 17 and Spring Boot 3.2.5 [2], PostgreSQL 17 [4] stores the data, and Spring Security with JWT [3], [5] protects access.

A. Problem Statement

Where customer, lead and sales data is managed manually or in several disconnected applications, the following problems typically occur:

- duplicated and inconsistent customer information;
- difficulty tracking leads and their requirements;
- inefficient management of sales opportunities;
- no central record of calls, meetings and other activities; and
- weak control over who can access business data.

MaximizeCRM addresses these problems with a centralized, secured web application.

B. Objectives

- Provide a centralized platform for customer relationship management.
- Maintain structured records of customers, leads and opportunities.
- Organize lead requirements, campaigns, products and activities.
- Provide secure, token-based user authentication using JWT.
- Expose RESTful APIs for frontend–backend communication.
- Store data persistently in PostgreSQL.
- Document the APIs through Swagger/OpenAPI.
- Keep the architecture modular and extensible.

C. Scope

The project covers user, customer, lead, lead-requirement, opportunity, campaign, product and activity management, together with authentication and API documentation. Advanced reporting, automated notifications, task scheduling and external integrations are left as future work.

D. Comparison with Manual Approaches

Table I contrasts the common spreadsheet-based way of working with the approach taken in MaximizeCRM.

TABLE I
MANUAL TRACKING VERSUS MAXIMIZECRM

Aspect	Spreadsheets	MaximizeCRM
Data location	Scattered files	One PostgreSQL database
Duplication	Common	Controlled by relational schema
Access control	File sharing	JWT-secured REST API
Activities	Ad hoc notes	Dedicated activity records
Pipeline view	Manual summaries	Live dashboard KPIs
Extensibility	Limited	Modular architecture

II. TECHNOLOGIES USED

Java 17 is the backend language. Its object-oriented model (classes, inheritance, encapsulation, abstraction) supports a modular code base.

Spring Boot 3.2.5 simplifies building production-ready Java applications through auto-configuration, dependency management and an embedded server [2]. It is used to build the REST services, handle HTTP requests, implement business logic, integrate with PostgreSQL and configure security.

PostgreSQL 17 is an open-source relational database [4]. It stores users, customers, leads, opportunities, campaigns, products and activities, and its relational model keeps related CRM records consistent.

Spring Security and JWT. Spring Security [3] provides authentication and authorization. A JSON Web Token [5] is a signed token that the backend issues after a successful login and validates on each protected request.

REST and OpenAPI. The frontend and backend communicate over REST using HTTP methods such as GET, POST, PUT and DELETE. Springdoc OpenAPI 2.5.0 generates interactive API documentation (Swagger UI) following the OpenAPI Specification [6], [7].

III. SYSTEM REQUIREMENTS

Table II lists the environment needed to develop and run MaximizeCRM.

TABLE II
HARDWARE AND SOFTWARE REQUIREMENTS

Item	Requirement
Processor / RAM	Intel Core i3 or equivalent / 4 GB
Storage / Network	10 GB free / internet for dependencies
Language	Java 17
Backend framework	Spring Boot 3.2.5
Database	PostgreSQL 17 (JDBC driver)
Build tool / IDE	Apache Maven / IntelliJ IDEA
Security	Spring Security, JWT
API documentation	Springdoc OpenAPI 2.5.0
Version control	Git and GitHub
Frontend	Separate MaximizeCRM web application

A. Functional Requirements

1. **Authentication:** authenticate users and restrict access to protected resources.
2. **Customer management:** maintain customer information in structured form.
3. **Lead management:** manage prospective customers and their details.
4. **Lead requirements:** record the requirements of each lead.
5. **Opportunities:** organize potential sales and track their stage.
6. **Campaigns:** maintain marketing and business campaign data.
7. **Products:** maintain product (inventory) information.
8. **Activities:** record activities and link them to CRM entities and products.
9. **API communication and documentation:** expose REST APIs and publish them through Swagger/OpenAPI.

B. Non-Functional Requirements

Security (authenticated access to all resources); *reliability* (consistent storage and retrieval); *maintainability* (modular frontend and backend); *scalability* (room for new modules); *usability* (a clear interface); and *performance* (efficient request and query handling).

IV. SYSTEM ARCHITECTURE AND DESIGN

MaximizeCRM follows a client–server architecture that separates presentation from application logic and data. Each layer can therefore be developed, tested and deployed independently.

A. Architectural Layers

The **presentation layer** is the frontend application. It displays data, accepts user input and calls the backend over HTTP. The **application layer** is the Spring Boot backend, which handles API requests, authentication, validation and business logic, and contains the security and CORS configuration. The **database layer** is PostgreSQL, which holds the records and their relationships. Fig. 1 shows the overall structure.

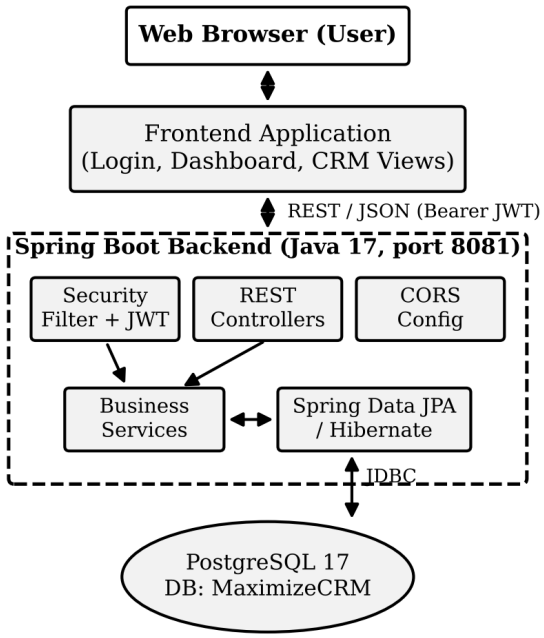


Fig. 1. Overall system architecture of MaximizeCRM.

B. Request Workflow

1. The user opens the web application and signs in.
2. The frontend sends an HTTP request to the backend.
3. Security filters validate the JWT where required.
4. The controller and service layer execute the operation.
5. PostgreSQL stores or retrieves the data via JPA/Hibernate.
6. The backend returns an HTTP response and the frontend renders it.

C. Modules

The system is organized into nine entities. The deployed interface presents them with dealership terminology, as mapped in Table III.

TABLE III
CRM ENTITIES AND THEIR INTERFACE LABELS

Entity	Table	Interface label
User	users	Sales Staff
Customer	customer	Buyers
Lead	lead	Potential Buyers
Lead requirement	lead_requirement	(per-lead needs)

Entity	Table	Interface label
Opportunity	opportunity	Sales Pipeline
Campaign	campaign	Dealership Campaigns
Product	product	Vehicle Inventory
Activity	activity	Activities & Test Drives
Activity product	activity_product	(activity–product link)

D. Use Cases

The primary actor is an authenticated sales staff member. The main use cases are signing in, viewing dashboard KPIs, and managing customers, leads, lead requirements, opportunities, campaigns, products and activities (Fig. 2).

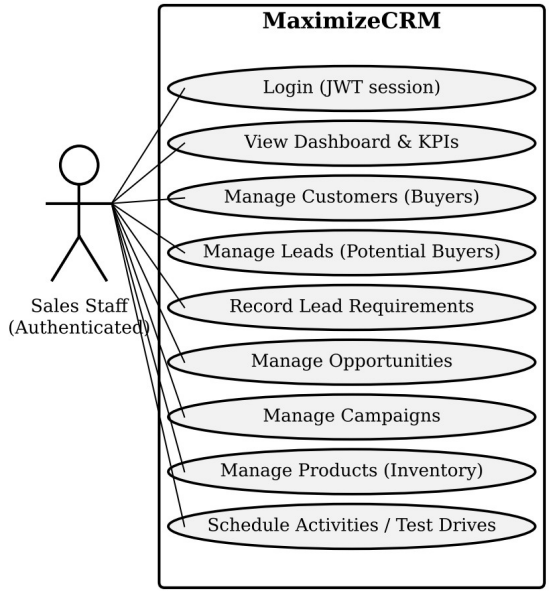


Fig. 2. Use case diagram for the authenticated sales staff actor.

E. Design Considerations

Several design decisions shape the system:

- **Separation of concerns:** presentation, application logic and storage are independent layers that communicate through defined interfaces.
- **API-first approach:** every feature is available as a REST endpoint documented in Swagger UI, so the frontend can be replaced or extended without backend changes.
- **Stateless authentication:** a JWT carries the information needed to authenticate each request, so the backend does not need to keep server-side sessions and can scale horizontally.
- **Relational integrity:** related records are stored in separate tables and linked by keys, which avoids duplicated customer data.
- **Extensibility:** a new CRM module needs an entity, its endpoints and a frontend view, without disturbing existing modules.

V. DATABASE DESIGN

The application uses PostgreSQL 17 with a database named *MaximizeCRM* and the *public* schema. Fig. 3 shows the nine tables created in the database.

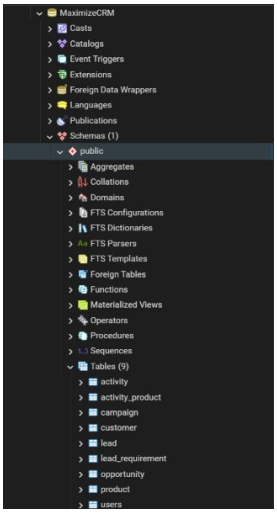


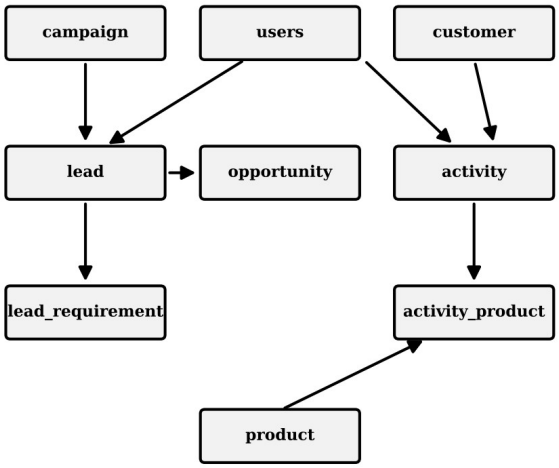
Fig. 3. Database tables in the MaximizeCRM schema (pgAdmin).

A. Entities

- **users** – application accounts used for login and account management.
- **customer** – existing customers (buyers).
- **lead** – prospective customers and enquiries, with status and source channel.
- **lead_requirement** – requirements attached to a lead.
- **opportunity** – potential sales and their value and stage.
- **campaign** – marketing and business campaigns.
- **product** – products such as vehicle models and stock.
- **activity** – calls, meetings and other interactions, with date and status.
- **activity_product** – association table linking activities and products.

B. Relationships

Leads own their requirement records, activities can reference several products through *activity_product*, and customers, users and campaigns are associated with leads and activities, as exercised by the relationship test records visible in the dashboard. Fig. 4 shows the main associations.



Parent → child links shown are indicative.

Fig. 4. Entity–relationship overview of the nine tables.

C. Database Connectivity

The backend connects to PostgreSQL through Spring Data JPA and the PostgreSQL JDBC driver. Hibernate

maps Java entity classes to tables. In development, Hibernate schema management initializes the tables; for production, controlled migrations (for example Flyway or Liquibase) are recommended.

VI. IMPLEMENTATION

A. Backend

The backend is a Maven-built Spring Boot 3.2.5 application on Java 17, running on port 8081 in development. Its main parts are: **controllers**, which handle HTTP requests; **entities/models**, which represent the data; **configuration classes** for application, security and CORS settings; **security components**, which authenticate requests; and the **database integration** layer.

B. Frontend

The frontend is maintained as a separate application and communicates with the backend only through REST APIs. Staff first reach the login page (Fig. 5), which restricts access to authorized personnel and states that a JWT session is used. After login, a dashboard (Fig. 6) shows live operational figures and a navigation menu for Vehicle Inventory, Buyers, Potential Buyers, Sales Pipeline, Activities & Test Drives, Dealership Campaigns and Sales Staff.

The dashboard (Fig. 6) summarizes the business at a glance: total buyers, active leads, open opportunities with the value of the active pipeline, and inventory, followed by a pipeline board (open deals, negotiation, deals won, deals lost), the most recent potential buyers with their status and channel, and upcoming activities such as meetings, calls and test drives.

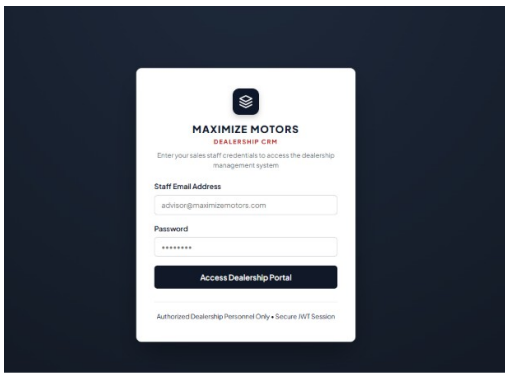


Fig. 5. Secure staff login page.

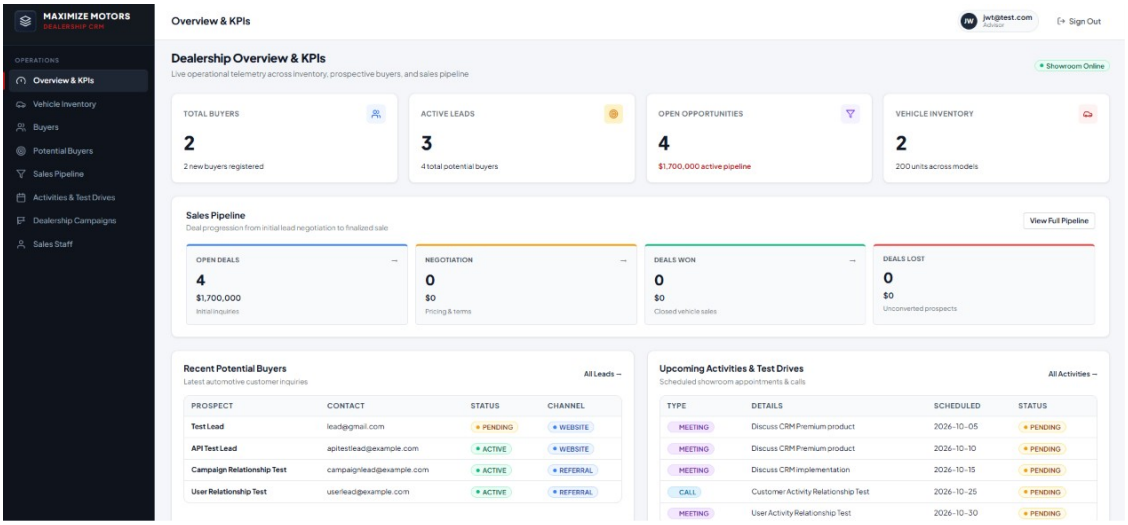


Fig. 6. Dealership overview and KPI dashboard of MaximizeCRM.

C. Security Implementation

Security is important because the application handles business and customer data. Token-based authentication follows these steps:

1. The user submits credentials from the login page.
2. The backend validates them against stored user data.
3. On success, a signed JWT is issued to the frontend.
4. The frontend sends the token with each protected request.
5. A security filter validates the token before access is granted.

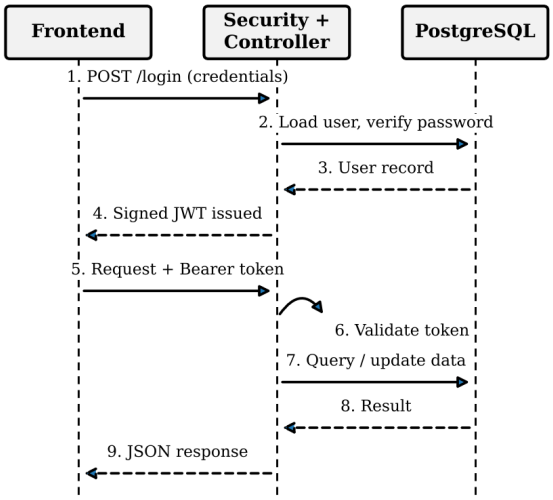


Fig. 7. JWT authentication and request sequence.

Passwords should be stored using a strong hashing algorithm, and the JWT signing secret should be kept

outside the source repository (for example in environment variables).

D. Cross-Origin Resource Sharing

Because the frontend and backend are separate applications, a CORS configuration defines which origins, methods and headers may call the backend. During development it allows the frontend to connect; in production only trusted origins should be permitted.

E. Build and Run

The backend is built with Maven and started as a standalone Spring Boot application. The datasource URL, database user and password, and the JWT signing secret are supplied through application configuration and environment variables rather than being committed to the repository. Once the backend is running on port 8081 and PostgreSQL is reachable, the frontend is started separately and pointed at the backend base URL, after which staff can sign in and use all modules.

VII. API DEVELOPMENT AND DOCUMENTATION

The backend exposes REST endpoints through which the frontend reads and modifies CRM data. Table IV summarizes the HTTP methods used; the exact set available depends on each controller.

TABLE IV
HTTP METHODS IN THE REST APIS

Method	Purpose
GET	Retrieve existing records
POST	Create new records (including login)
PUT / PATCH	Update records fully or partially
DELETE	Remove records

Springdoc OpenAPI 2.5.0 generates Swagger UI, which lets developers list endpoints, inspect request and response structures, supply parameters and execute calls against the running backend [6], [7]. This simplifies backend testing and gives frontend developers a reliable contract to build against.

VIII. TESTING AND VERIFICATION

Testing covered the backend build, database connectivity, authentication, API behavior and frontend integration. Table V summarizes the objectives.

TABLE V
TESTING SUMMARY

Area	Verification objective
Build	Maven compiles and resolves dependencies
Database	Connection, table mapping, persistence
Authentication	Valid/invalid login; token validation
API	Status codes, payloads, error handling
Frontend	Login, dashboard data and navigation
Security	Protected resources reject missing or invalid tokens

Build and database. A successful Maven build confirms compilation and dependency resolution only; it does not prove that workflows function. Database checks confirm that the application connects to PostgreSQL, that the nine tables are created (Fig. 3) and that records are stored and retrieved.

Authentication. Scenarios to verify are login with valid and invalid credentials, access to protected resources without a token, with a valid token, and with an expired or invalid token.

API and integration. Swagger UI is used to execute requests and inspect responses. The running dashboard (Fig. 6) shows data returned by the backend, including records named for API and relationship tests (for example “API Test Lead”, “Campaign Relationship Test” and “User Relationship Test”), which indicates that the create and association operations were exercised end to end.

Comprehensive automated unit, integration and security tests remain further quality-assurance work.

IX. RESULTS AND DISCUSSION

The working prototype was exercised with a small test dataset. Table VI lists the key figures shown on the dashboard in Fig. 6, which are computed from data stored in PostgreSQL and returned through the REST APIs.

TABLE VI
DASHBOARD INDICATORS FROM THE TEST DATASET

Indicator	Value shown
Total buyers	2
Potential buyers	4 (3 active)
Open opportunities	4
Active pipeline value	$1,700,000
Vehicle inventory	2 models, 200 units
Negotiation / won / lost	0 / 0 / 0

The pipeline board shows all four open deals in the initial-inquiry stage, and the recent potential buyers list shows one pending and three active leads arriving through the website and referral channels. The upcoming activities panel lists scheduled meetings and calls with a pending status, so staff can see what is due next without opening each record.

The results show that data created in one module is reflected consistently in aggregate views: leads feed the lead and pipeline counters, activities feed the schedule panel, and products feed the inventory figure. The dataset is intentionally small and consists of test records, so the figures demonstrate correct behavior of the integrated system rather than real sales performance. Load testing and evaluation on realistic data volumes remain future work.

X. DEPLOYMENT AND VERSION CONTROL

The frontend and backend are deployed independently. The backend needs Java 17 and a PostgreSQL instance; the frontend needs its own build and hosting environment. The source code is maintained with Git in a GitHub repository [8], which preserves project history and tracks changes. Table VII lists the project details.

TABLE VII
PROJECT AND DEPLOYMENT DETAILS

Item	Details
Project	MaximizeCRM
Repository	github.com/Agrima0408/MaximizeCRM

Item	Details
Backend port (dev)	8081
Frontend URL	[Insert live frontend URL]
Backend URL	[Insert live backend URL]
Swagger URL	[Insert deployed Swagger URL]

If the application is not yet hosted, state “local development environment – deployment pending” instead of a URL. For production, HTTPS, protected environment variables, regular database backups and restricted CORS origins should be configured.

XI. LIMITATIONS AND FUTURE SCOPE

A. Current Limitations

The current version establishes the core modules and architecture. It can be strengthened with comprehensive automated testing, advanced reporting and analytics, automated follow-up notifications, detailed role-based access control, richer pipeline visualization, and production deployment with performance evaluation.

B. Future Enhancements

- **Analytics:** dashboards for lead conversion, campaign performance and sales trends.
- **Automated follow-ups:** reminders and notifications for activities.
- **Role-based access control:** separate permissions for administrators, managers and staff.
- **Email integration:** manage customer communication inside the CRM.
- **Search and filtering:** faster location of customers, leads and activities.
- **Import/export:** CSV and Excel support.
- **Performance:** indexing, pagination, query tuning and caching.
- **Cloud deployment:** improved availability and scalability.

XII. CONCLUSION

MaximizeCRM is a web-based CRM that provides one platform for customer-related information and sales activities. A separate frontend communicates through REST APIs with a Spring Boot 3.2.5 backend on Java 17, PostgreSQL 17 stores the data, and Spring Security with JWT secures access. The nine-table schema covers users, customers, leads, lead requirements, opportunities, campaigns, products and activities, and the dealership dashboard shows how these entities combine into a working sales view with KPIs, a pipeline board and a test-drive schedule.

Its modular architecture and structured database make the system easy to extend with analytics, automated follow-ups, finer-grained access control and external integrations, demonstrating a practical use of modern Java web technologies for an extensible CRM.

REFERENCES

[1] F. Buttle and S. Maklan, *Customer Relationship Management: Concepts and Technologies*, 4th ed. London, U.K.: Routledge, 2019.

[2] VMware, “Spring Boot Reference Documentation, Version 3.2.5.” [Online]. Available: https://docs.spring.io/spring-boot/docs/3.2.5/reference/html/

[3] Spring, “Spring Security Reference.” [Online]. Available: https://docs.spring.io/spring-security/reference/

[4] PostgreSQL Global Development Group, “PostgreSQL 17 Documentation.” [Online]. Available: https://www.postgresql.org/docs/17/

[5] M. Jones, J. Bradley and N. Sakimura, “JSON Web Token (JWT),” IETF RFC 7519, May 2015. [Online]. Available: https://www.rfc-editor.org/rfc/rfc7519

[6] OpenAPI Initiative, “OpenAPI Specification.” [Online]. Available: https://spec.openapis.org/oas/latest.html

[7] Springdoc, “springdoc-openapi Documentation.” [Online]. Available: https://springdoc.org/

[8] Agrima0408, “MaximizeCRM,” GitHub repository. [Online]. Available: https://github.com/Agrima0408/MaximizeCRM